About this Book

This book introduces matrices through high school coordinate geometry. This approach makes it easier for beginners to learn Python for scientific computing. All problems in the book are from NCERT mathematics textbooks from Class 9-12. Exercises are from CBSE board exam papers.

The content is sufficient for industry jobs and covers nearly all matrix prerequisites for machine learning. There is no copyright, so readers are free to print and share.

License: https://creativecommons.org/licenses/by-sa/3.0/
and
https://www.gnu.org/licenses/fdl-1.3.en.html

First manual appeared in January 2018
First edition published on July 10, 2024

In this edition, some incorrect solutions were removed. All figures redrawn. More problems added.

CONTENTS

1 GATE CS1 2025 3
1.1 Formulae 3
1.2 Problems 9

1. GATE CS1 2025

A. Formulae

1.1.1 Foundational Definition of the Unilateral Z-Transform:

$$\mathcal{Z}\{x(n)\} = X(z) = \sum_{n=0}^{\infty} x(n) z^{-n} \tag{1.1.1.1}$$

1.1.2 Probability Mass Function (PMF) and Z-Transform of Discrete Random Variables:

$$M_Y(Z) = E[Z^{-Y}] = \sum_{k} P_Y(k) Z^{-k} \tag{1.1.2.1}$$

1.1.3 Z-Transform of Sum of Independent Random Variables $Y = \sum_{i=1}^{n} Y_i$:

$$M_Y(Z) = \prod_{i=1}^{n} M_{Y_i}(Z) \tag{1.1.3.1}$$

Solution: For n independent discrete random variables $Y_1, Y_2, \ldots, Y_n$, the sum is given by $Y = \sum_{i=1}^{n} Y_i$. Applying the foundational Z-transform definition from (1.1.1.1):

$$M_Y(Z) = E[Z^{-Y}] = E\left[Z^{-\sum_{i=1}^{n} Y_i}\right] = E\left[\prod_{i=1}^{n} Z^{-Y_i}\right] \tag{1.1.3.2}$$

Since the random variables $Y_1, Y_2, \ldots, Y_n$ are mutually independent, the expectation of their product equals the product of their individual expectations:

$$M_Y(Z) = \prod_{i=1}^{n} E[Z^{-Y_i}] = \prod_{i=1}^{n} M_{Y_i}(Z) \tag{1.1.3.3}$$

1.1.4 Bayes' Theorem:

$$\Pr(E_1|A) = \frac{\Pr(E_1)\Pr(A|E_1)}{\sum_{i=1}^{2} \Pr(E_i)\Pr(A|E_i)} \tag{1.1.4.1}$$

Solution: Bayes' theorem relates the conditional probability of event E_1 given evidence A to the prior probabilities $\Pr(E_1)$ and $\Pr(E_2)$ and the likelihoods $\Pr(A|E_1)$ and $\Pr(A|E_2)$:

$$\Pr(E_1|A) = \frac{\Pr(A \cap E_1)}{\Pr(A)} \tag{1.1.4.2}$$

$$= \frac{\Pr(E_1)\Pr(A|E_1)}{\Pr(E_1)\Pr(A|E_1) + \Pr(E_2)\Pr(A|E_2)} \tag{1.1.4.3}$$

$$= \frac{\Pr(E_1)\Pr(A|E_1)}{\sum_{i=1}^{2} \Pr(E_i)\Pr(A|E_i)} \tag{1.1.4.4}$$

1.1.5 Composition of Functions and Surjectivity (Onto Property):

$$(f \circ g)(x) = f(g(x)), \quad \forall x \in A \tag{1.1.5.1}$$

1.1.6 Shortest Path Inequality in Subgraphs and Spanning Trees: If T is a minimum spanning tree of graph G, then for any vertices $u, v \in V(G)$:

$$d_1(u, v) \leq d_2(u, v) \tag{1.1.6.1}$$

where $d_1(u, v)$ is the shortest path in G and $d_2(u, v)$ is the shortest path in T.

1.1.7 Z-Transform Property of $na^n u(n)$:

$$\mathcal{Z}\{na^n u(n)\} = \frac{az^{-1}}{(1 - az^{-1})^2} \tag{1.1.7.1}$$

Solution: From the foundational definition of the unilateral Z-transform in (1.1.1.1), the Z-transform of the sequence $x(n) = na^n u(n)$ is given by the infinite series summation:

$$\mathcal{Z}\{na^n u(n)\} = \sum_{n=0}^{\infty} na^n z^{-n} \tag{1.1.7.2}$$

$$= \sum_{n=0}^{\infty} n(az^{-1})^n \tag{1.1.7.3}$$

Let $x = az^{-1}$. The summation can be expressed as:

$$S = \sum_{n=0}^{\infty} nx^n \tag{1.1.7.4}$$

Taking the standard geometric series identity, which converges for $|x| < 1$:

$$\sum_{n=0}^{\infty} x^n = \frac{1}{1 - x} \tag{1.1.7.5}$$

Differentiating both sides with respect to x:

$$\frac{d}{dx}\left(\sum_{n=0}^{\infty} x^n\right) = \frac{d}{dx}\left(\frac{1}{1 - x}\right) \tag{1.1.7.6}$$

$$\sum_{n=0}^{\infty} nx^{n-1} = \frac{1}{(1 - x)^2} \tag{1.1.7.7}$$

Multiplying both sides by x to match the power of nx^n:

$$\sum_{n=0}^{\infty} nx^n = \frac{x}{(1 - x)^2} \tag{1.1.7.8}$$

Finally, substituting back $x = az^{-1}$ into the derived expression yields the closed-

form Z-transform:

$$\mathcal{Z}\{na^n u(n)\} = \frac{az^{-1}}{(1-az^{-1})^2} \tag{1.1.7.9}$$

1.1.8 Binomial Coefficient Symmetry Identity:

$$\binom{n}{k} = \binom{n}{n-k} \tag{1.1.8.1}$$

Solution: Using the factorial definition of the binomial coefficient for $0 \leq k \leq n$:

$$\binom{n}{k} = \frac{n!}{k!(n-k)!} \tag{1.1.8.2}$$

Substituting $(n-k)$ in place of k:

$$\binom{n}{n-k} = \frac{n!}{(n-k)!(n-(n-k))!} = \frac{n!}{(n-k)!k!} \tag{1.1.8.3}$$

Since multiplication in the denominator is commutative:

$$\binom{n}{k} = \binom{n}{n-k} \tag{1.1.8.4}$$

1.1.9 Negative Binomial Coefficient Identity:

$$\binom{-n}{k} = (-1)^k \binom{n+k-1}{k} \tag{1.1.9.1}$$

Solution: Extending the binomial coefficient definition to negative integer upper index $-n$ and non-negative integer k:

$$\binom{-n}{k} = \frac{(-n)(-n-1)(-n-2)\dots(-n-k+1)}{k!} \tag{1.1.9.2}$$

Factoring out (-1) from each of the k factors in the numerator:

$$(-n)(-n-1)(-n-2)\dots(-n-k+1) = (-1)^k n(n+1)(n+2)\dots(n+k-1) \tag{1.1.9.3}$$

$$= (-1)^k \frac{(n+k-1)!}{(n-1)!} \tag{1.1.9.4}$$

Substituting this back into the denominator $k!$:

$$\binom{-n}{k} = (-1)^k \frac{(n+k-1)!}{(n-1)!k!} \tag{1.1.9.5}$$

$$= (-1)^k \binom{n+k-1}{k} \tag{1.1.9.6}$$

1.1.10 Generalized Binomial Expansion Series in Summation Form:

$$(1-x)^{-n} = \sum_{k=0}^{\infty} \binom{n+k-1}{k} x^k, \quad \text{for } |x| < 1 \tag{1.1.10.1}$$

Solution: By Taylor series expansion for any real power $-n$ and $|x| < 1$:

$$(1-x)^{-n} = \sum_{k=0}^{\infty} \binom{-n}{k} (-x)^k \tag{1.1.10.2}$$

Using the negative binomial coefficient identity from (1.1.9.1):

$$\binom{-n}{k}(-x)^k = \left[(-1)^k \binom{n+k-1}{k}\right](-1)^k x^k \tag{1.1.10.3}$$

$$= (-1)^{2k} \binom{n+k-1}{k} x^k \tag{1.1.10.4}$$

$$= \binom{n+k-1}{k} x^k \tag{1.1.10.5}$$

Substituting this back yields the closed-form summation:

$$(1-x)^{-n} = \sum_{k=0}^{\infty} \binom{n+k-1}{k} x^k \tag{1.1.10.6}$$

1.1.11 Inverse Z-Transform of $\dfrac{1}{(1-az^{-1})^n}$ for Integer $n > 0$:

$$\mathcal{Z}^{-1}\left\{\frac{1}{(1-az^{-1})^n}\right\} = \binom{n+k-1}{n-1} a^k u(k) \tag{1.1.11.1}$$

Solution: Writing the fraction in negative exponent form:

$$\frac{1}{(1-az^{-1})^n} = (1-az^{-1})^{-n} \tag{1.1.11.2}$$

Letting $x = az^{-1}$, where $|az^{-1}| < 1$, we apply the generalized binomial series expansion from (1.1.10.1):

$$(1-az^{-1})^{-n} = \sum_{k=0}^{\infty} \binom{n+k-1}{k} (az^{-1})^k = \sum_{k=0}^{\infty} \binom{n+k-1}{k} a^k z^{-k} \tag{1.1.11.3}$$

Using the symmetry property of binomial coefficients from (1.1.8.1), we have $\binom{n+k-1}{k} = \binom{n+k-1}{(n+k-1)-k} = \binom{n+k-1}{n-1}$:

$$\frac{1}{(1-az^{-1})^n} = \sum_{k=0}^{\infty} \binom{n+k-1}{n-1} a^k z^{-k} \tag{1.1.11.4}$$

Taking the inverse Z-transform by comparing with (1.1.1.1):

$$\mathcal{Z}^{-1}\left\{\frac{1}{(1-az^{-1})^n}\right\} = \binom{n+k-1}{n-1}a^k u(k) \tag{1.1.11.5}$$

Special cases for $n = 1, 2, 3$:

- $n = 1$: $\mathcal{Z}^{-1}\left\{\frac{1}{1-az^{-1}}\right\} = \binom{k}{0}a^k u(k) = a^k u(k)$
- $n = 2$: $\mathcal{Z}^{-1}\left\{\frac{1}{(1-az^{-1})^2}\right\} = \binom{k+1}{1}a^k u(k) = (k+1)a^k u(k)$
- $n = 3$: $\mathcal{Z}^{-1}\left\{\frac{1}{(1-az^{-1})^3}\right\} = \binom{k+2}{2}a^k u(k) = \frac{(k+1)(k+2)}{2}a^k u(k)$

1.1.12 System of Linear Equations Consistency Analysis: For a linear system $\mathbf{Ax} = \mathbf{b}$, the existence and uniqueness of solutions are determined by rank conditions on $\mathbf{A}$ and the augmented matrix $[\mathbf{A} \mid \mathbf{b}]$:

$$\begin{cases} \text{Unique Solution:} & \text{rank}(\mathbf{A}) = \text{rank}([\mathbf{A} \mid \mathbf{b}]) = n \\ \text{Infinite Solutions:} & \text{rank}(\mathbf{A}) = \text{rank}([\mathbf{A} \mid \mathbf{b}]) < n \\ \text{No Solution:} & \text{rank}(\mathbf{A}) \neq \text{rank}([\mathbf{A} \mid \mathbf{b}]) \end{cases} \tag{1.1.12.1}$$

1.1.13 Number of Strings over Alphabet Size k without Adjacent Duplicate Symbols:

$$N_{\text{no_consec}} = k(k-1)^{n-1} \tag{1.1.13.1}$$

Solution: For a string of length n defined over an alphabet of size k:

- The first character can be chosen in k different ways.
- For each subsequent position from 2 to n, the character must be distinct from the immediately preceding character, giving $(k-1)$ choices per position.

By the fundamental counting principle, the total number of such strings is:

$$N_{\text{no_consec}} = k \times \underbrace{(k-1) \times (k-1) \times \cdots \times (k-1)}_{n-1 \text{ times}} = k(k-1)^{n-1} \tag{1.1.13.2}$$

1.1.14 Boundary Continuity and Differentiability Conditions for Piecewise Functions:

$$\lim_{x \to x_0^-} f(x) = \lim_{x \to x_0^+} f(x) = f(x_0) \tag{1.1.14.1}$$

$$\lim_{x \to x_0^-} f'(x) = \lim_{x \to x_0^+} f'(x) \tag{1.1.14.2}$$

1.1.15 Total Inversions (Swaps) in a Reverse-Sorted Array of Size n:

$$\text{Swaps} = \frac{n(n-1)}{2} \tag{1.1.15.1}$$

Solution: For an array of size n sorted in strictly descending order, every adjacent pair satisfies $A[j] > A[j+1]$. The total number of swap operations is the sum of comparisons across all passes:

$$\begin{aligned} \text{Swaps} &= \sum_{i=0}^{n-2} (n-i-1) && (1.1.15.2)\\ &= (n-1) + (n-2) + \cdots + 2 + 1 && (1.1.15.3)\\ &= \frac{n(n-1)}{2} && (1.1.15.4) \end{aligned}$$

1.1.16 Matrix Characteristic Equation and Eigenvalues of Powers: The eigenvalues λ of $\mathbf{A}$ satisfy $\det(\mathbf{A} - \lambda\mathbf{I}) = 0$. For any positive integer k:

$$\mathbf{A}\mathbf{x} = \lambda\mathbf{x} \implies \mathbf{A}^k\mathbf{x} = \lambda^k\mathbf{x} \tag{1.1.16.1}$$

Solution: Given the eigenvalue definition $\mathbf{A}\mathbf{x} = \lambda\mathbf{x}$, multiplying by $\mathbf{A}$ successively k times gives:

$$\begin{aligned} \mathbf{A}^2\mathbf{x} &= \mathbf{A}(\mathbf{A}\mathbf{x}) && (1.1.16.2)\\ &= \mathbf{A}(\lambda\mathbf{x}) && (1.1.16.3)\\ &= \lambda(\mathbf{A}\mathbf{x}) && (1.1.16.4)\\ &= \lambda^2\mathbf{x} && (1.1.16.5) \end{aligned}$$

Extending this result by mathematical induction for any positive integer k:

$$\mathbf{A}^k\mathbf{x} = \lambda^k\mathbf{x} \tag{1.1.16.6}$$

1.1.17 Minimum Height of Breadth-First-Search (BFS) Tree given Graph Diameter D:

$$h(T) \geq \left\lceil \frac{D}{2} \right\rceil \tag{1.1.17.1}$$

1.1.18 Basic Block Leader Criteria in Compiler Intermediate Code:

$$\text{Leaders} = \{\text{First Instruction}\} \cup \{\text{Targets of Jumps}\} \cup \{\text{Instructions Following Jumps}\} \tag{1.1.18.1}$$

1.1.19 Continuous Probability Density Function (PDF) Normalization:

$$\int_{-\infty}^{\infty} P(x)\,dx = 1 \tag{1.1.19.1}$$

1.1.20 Cumulative Distribution Function (CDF) and Range Probability:

$$F_X(x) = \int_{-\infty}^{x} P(t)\,dt, \quad P(a \leq X \leq b) = F_X(b) - F_X(a) = \int_a^b P(x)\,dx \tag{1.1.20.1}$$

Solution: The Cumulative Distribution Function (CDF) $F_X(x)$ accumulates the probability density from the lower support limit up to x. The probability that a continuous random variable X falls within the interval $[a, b]$ is obtained by evaluating the definite integral of the probability density function $P(x)$, which equals the difference in CDF values:

$$P(a \leq X \leq b) = \int_a^b P(x)\,dx \tag{1.1.20.2}$$

$$= \int_{-\infty}^b P(x)\,dx - \int_{-\infty}^a P(x)\,dx \tag{1.1.20.3}$$

$$= F_X(b) - F_X(a) \tag{1.1.20.4}$$

1.1.21 Open Addressing Double Hashing Probe Sequence:

$$h(k, i) = [h_1(k) + i \cdot h_2(k)] \bmod m \tag{1.1.21.1}$$

Solution: In open addressing with double hashing, when a collision occurs at the primary hash location $h_1(k)$, subsequent probe locations are determined by adding a step size governed by the second auxiliary hash function $h_2(k)$ multiplied by the probe index i:

$$h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m \tag{1.1.21.2}$$

where m is the hash table size and $i \in \{0, 1, \ldots, m-1\}$. To ensure all slots can be probed, $h_2(k)$ must be relatively prime to m.

B. Problems

1.2.1 A fair six-faced dice, with the faces labelled '1', '2', '3', '4', '5', and '6', is rolled thrice. What is the probability of rolling '6' exactly once? (CS1 2025)

a) $\frac{75}{216}$ b) $\frac{1}{6}$ c) $\frac{1}{18}$ d) $\frac{25}{216}$

Solution: Step 1: Finding the probability mass function (PMF) of a single dice roll The sample space is $S = \{1, 2, 3, 4, 5, 6\}$. Let us define a variable Y_1 as follows:

$$Y_1 = \begin{cases} 1, & X_1 = 6 \\ 0, & X_1 \neq 6 \end{cases} \tag{1.2.1.1}$$

Now, the PMF is defined as:

$$P_{Y_1}(k) = \begin{cases} \frac{1}{6}, & k = 1 \\ \frac{5}{6}, & k = 0 \end{cases} \tag{1.2.1.2}$$

Let p, q be $\frac{1}{6}$ and $\frac{5}{6}$ respectively.

Step 2: Applying Z-Transform to find the required PMF for Y
Let us define Y_2 and Y_3 as the random variables for the 2nd and 3rd dice roll, respectively. Let Y denote the random variable for the combined triple dice roll, $Y = Y_1 + Y_2 + Y_3$.
From (1.1.2.1), writing the Z-Transform for Y_1:

$$M_{Y_1}(Z) = E[Z^{-Y_1}] \tag{1.2.1.3}$$
$$= \sum P_{Y_1}(k) Z^{-k} \tag{1.2.1.4}$$
$$= P_{Y_1}(1) Z^{-1} + P_{Y_1}(0) Z^0 \tag{1.2.1.5}$$
$$= pZ^{-1} + q \tag{1.2.1.6}$$

Since Y_1, Y_2, Y_3 are independent events, using (1.1.3.1), we can write $M_Y(Z) = M_{Y_1}(Z)\, M_{Y_2}(Z)\, M_{Y_3}(Z)$.
Therefore,

$$M_Y(Z) = (pZ^{-1} + q)^3 \tag{1.2.1.7}$$
$$= \sum_{k=0}^{3} \binom{3}{k} q^{3-k} (pZ^{-1})^k \tag{1.2.1.8}$$
$$= \sum P_Y(k) Z^{-k} \tag{1.2.1.9}$$

Comparing coefficients, the PMF of Y is:

$$P_Y(k) = \binom{3}{k} q^{3-k} p^k \tag{1.2.1.10}$$

Step 3: Calculating required probability
We need the situation where p appears exactly once ($k = 1$). The required term is:

$$P_Y(1) = \binom{3}{1} p^1 q^2 \tag{1.2.1.11}$$
$$= 3 \cdot \left(\frac{1}{6}\right) \left(\frac{5}{6}\right)^2 \tag{1.2.1.12}$$
$$= \frac{75}{216} \tag{1.2.1.13}$$

Therefore, the required answer is $\frac{75}{216}$.
The following code gives the following figure (Fig. 1.2.1.1).

codes/pmf_plot.py

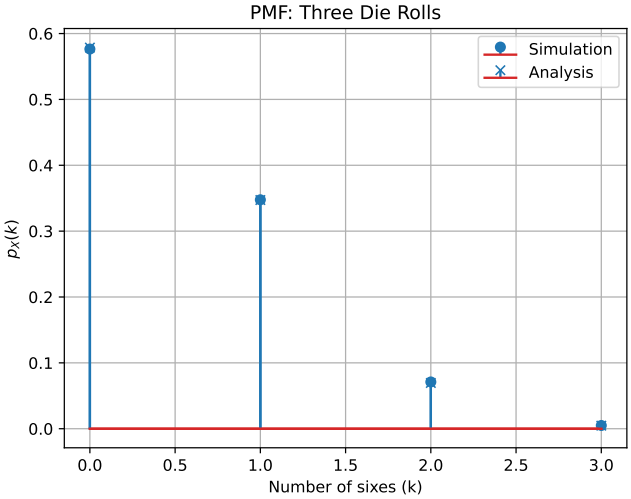


Fig. 1.2.1.1: Probability mass function for three die rolls

1.2.2 A shop has 4 distinct flavors of ice-cream. One can purchase any number of scoops of any flavor. The order in which the scoops are purchased is inconsequential. If one wants to purchase 3 scoops of ice-cream, in how many ways can one make that purchase? (CS1 2025)

a) 4 b) 20 c) 24 d) 48

Solution: Let F be the event that the chosen coin is the fake coin, and R be the event that it is a regular coin. The prior probabilities of selecting each coin are:

$$P(F) = \frac{1}{5} \tag{1.2.2.1}$$

$$P(R) = \frac{4}{5} \tag{1.2.2.2}$$

Let H_2 be the event of getting two heads when the chosen coin is tossed twice. The probability of getting two heads with a regular coin is:

$$P(H_2 \mid R) = \left(\frac{1}{2}\right) \times \left(\frac{1}{2}\right) = \frac{1}{4} \tag{1.2.2.3}$$

The probability of getting two heads with the fake coin is:

$$P(H_2 \mid F) = 1 \times 1 = 1 \tag{1.2.2.4}$$

Using Bayes' theorem (1.1.4.1):

$$P(F \mid H_2) = \frac{P(F)P(H_2 \mid F)}{P(F)P(H_2 \mid F) + P(R)P(H_2 \mid R)} \tag{1.2.2.5}$$

$$= \frac{\frac{1}{5} \times 1}{\left(\frac{1}{5} \times 1\right) + \left(\frac{4}{5} \times \frac{1}{4}\right)} \tag{1.2.2.6}$$

$$= 0.50 \tag{1.2.2.7}$$

Therefore, the probability that the coin chosen is the fake coin is 0.50.

1.2.3 $g(\cdot)$ is a function from A to B, $f(\cdot)$ is a function from B to C, and their composition defined as $f(g(\cdot))$ is a mapping from A to C. If $f(\cdot)$ and $f(g(\cdot))$ are onto (surjective) functions, which ONE of the following is TRUE about the function $g(\cdot)$? (CS1 2025)

a) $g(\cdot)$ must be an onto (surjective) function.
b) $g(\cdot)$ must be a one-to-one (injective) function.
c) $g(\cdot)$ must be a bijective function, that is, both one-to-one and onto.
d) $g(\cdot)$ is not required to be a one-to-one or onto function.

Solution: From (1.1.5.1), if $f \circ g : A \to C$ is onto, every element in C has a pre-image in A. This guarantees that f must be onto, but g does not need to cover all of B if f maps multiple elements of B to the same element in C. Thus, $g(\cdot)$ is not required to be one-to-one or onto.

1.2.4 Let G be any undirected graph with positive edge weights, and T be a minimum spanning tree of G. For any two vertices, u and v, let $d_1(u, v)$ and $d_2(u, v)$ be the shortest distances between u and v in G and T, respectively. Which ONE of the options is CORRECT for all possible G, T, u and v? (CS1 2025)

a) $d_1(u, v) = d_2(u, v)$
b) $d_1(u, v) \leq d_2(u, v)$
c) $d_1(u, v) \geq d_2(u, v)$
d) $d_1(u, v) \neq d_2(u, v)$

Solution: Let $d_1(u, v)$ and $d_2(u, v)$ be the shortest distances between u and v in G and T, respectively. By (1.1.6.1), since every path in T is a valid path in G, but G may contain additional edges offering shorter routes:

$$d_1(u, v) \leq d_2(u, v) \tag{1.2.4.1}$$

Therefore, option (b) is correct.

1.2.5 Consider the following recurrence relation: $T(n) = 2T(n-1) + n2^n$ for $n > 0$, $T(0) = 1$. Which ONE of the following options is CORRECT? (CS1 2025)

a) $T(n) = \Theta(n^2 2^n)$
b) $T(n) = \Theta(n2^n)$
c) $T(n) = \Theta((\log n)^2 2^n)$
d) $T(n) = \Theta(4^n)$

Solution: Let $x(n) = T(n)u(n)$ be a causal sequence such that $x(n) = 0$ for $n < 0$ and $x(0) = 1$. The given recurrence relation $T(n) - 2T(n-1) = n2^n$ for $n > 0$ with initial condition $T(0) = 1$ can be expressed using the unit impulse function $\delta(n)$ as:

$$x(n) - 2x(n-1) = n2^n u(n) + \delta(n) \tag{1.2.5.1}$$

Taking the Z-transform on both sides:

$$X(z) - 2z^{-1}X(z) = \mathcal{Z}\{n2^n u(n)\} + \mathcal{Z}\{\delta(n)\} \tag{1.2.5.2}$$

From (1.1.7.1), $\mathcal{Z}\{na^n u(n)\} = \frac{az^{-1}}{(1-az^{-1})^2}$ for $a = 2$, and $\mathcal{Z}\{\delta(n)\} = 1$:

$$X(z)(1 - 2z^{-1}) = \frac{2z^{-1}}{(1-2z^{-1})^2} + 1 \tag{1.2.5.3}$$

$$\implies X(z) = \frac{1}{1-2z^{-1}} + \frac{2z^{-1}}{(1-2z^{-1})^3} \tag{1.2.5.4}$$

$$= \frac{1 - 2z^{-1} + 4z^{-2}}{(1-2z^{-1})^3} \tag{1.2.5.5}$$

Applying partial fraction expansion:

$$X(z) = \frac{1}{1-2z^{-1}} - \frac{1}{(1-2z^{-1})^2} + \frac{2}{(1-2z^{-1})^3} \tag{1.2.5.6}$$

Taking the inverse Z-transform using (1.1.11.1):

$$x(n) = 2^n u(n) - (n+1)2^n u(n) + \frac{(n+1)(n+2)}{2} 2^n u(n) \tag{1.2.5.7}$$

$$= 2^n \left[1 - (n+1) + \frac{n^2 + 3n + 2}{2}\right] u(n) \tag{1.2.5.8}$$

$$= 2^n \left[\frac{n^2 + n + 2}{2}\right] u(n) \tag{1.2.5.9}$$

$$= 2^{n-1}(n^2 + n + 2)u(n) \tag{1.2.5.10}$$

Since $T(n) = x(n)$ for $n \geq 0$:

$$T(n) = 2^{n-1}(n^2 + n + 2) \tag{1.2.5.11}$$

$$= \Theta(n^2 2^n) \tag{1.2.5.12}$$

Therefore, $T(n) = \Theta(n^2 2^n)$.
The following code gives the following figure (Fig. 1.2.5.1).

codes/compare.py

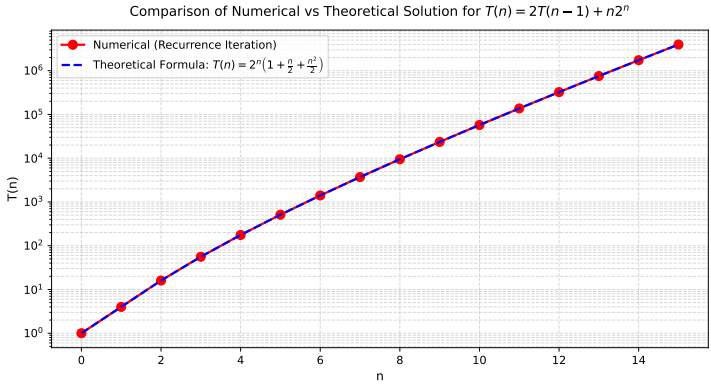


Fig. 1.2.5.1: Recurrence relation growth analysis

1.2.6 Consider the given system of linear equations for variables x and y, where k is a real-valued constant:

$$x + ky = 1$$
$$kx + y = -1$$

Which of the following option(s) is/are CORRECT? (CS1 2025)

a) There is exactly one value of k for which the above system of equations has no solution.
b) There exist an infinite number of values of k for which the system of equations has no solution.
c) There exists exactly one value of k for which the system of equations has exactly one solution.
d) There exists exactly one value of k for which the system of equations has an infinite number of solutions.

Solution: The given system of linear equations can be written in matrix form $\mathbf{Ax} = \mathbf{b}$ as:

$$\begin{bmatrix} 1 & k \\ k & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 1 \\ -1 \end{bmatrix} \tag{1.2.6.1}$$

Forming the augmented matrix $[\mathbf{A} \mid \mathbf{b}]$:

$$[\mathbf{A} \mid \mathbf{b}] = \left[\begin{array}{cc|c} 1 & k & 1 \\ k & 1 & -1 \end{array}\right] \tag{1.2.6.2}$$

Applying elementary row operation $R_2 \to R_2 - kR_1$ to reduce the matrix to row echelon form:

$$[\mathbf{A} \mid \mathbf{b}] \xrightarrow{R_2 \to R_2 - kR_1} \left[\begin{array}{cc|c} 1 & k & 1 \\ 0 & 1-k^2 & -1-k \end{array}\right] \tag{1.2.6.3}$$

Let $n = 2$ be the number of variables. From (1.1.12.1), we analyze the rank of the coefficient matrix $\text{rank}(\mathbf{A})$ and the augmented matrix $\text{rank}([\mathbf{A} \mid \mathbf{b}])$:
1. Condition for Unique Solution:
For a unique solution, $\text{rank}(\mathbf{A}) = \text{rank}([\mathbf{A} \mid \mathbf{b}]) = n = 2$.
This requires the pivot position in the second row to be non-zero:

$$1 - k^2 \neq 0 \implies k^2 \neq 1 \tag{1.2.6.4}$$

$$\implies k \neq 1 \quad \text{and} \quad k \neq -1 \tag{1.2.6.5}$$

Thus, a unique solution exists for infinitely many values of k ($k \in \mathbb{R} \setminus \{1, -1\}$).
2. Condition for Infinite Solutions:
For infinitely many solutions, $\text{rank}(\mathbf{A}) = \text{rank}([\mathbf{A} \mid \mathbf{b}]) < n = 2$, which means $\text{rank}(\mathbf{A}) = \text{rank}([\mathbf{A} \mid \mathbf{b}]) = 1$.
This requires both entries in the second row to vanish simultaneously:

$$1 - k^2 = 0 \quad \text{and} \quad -1 - k = 0 \tag{1.2.6.6}$$

From $-1-k = 0 \implies k = -1$. Substituting $k = -1$ into $1-k^2$ gives $1-(-1)^2 = 0$.
At $k = -1$, the row echelon form becomes:

$$\begin{bmatrix} 1 & -1 & | & 1 \\ 0 & 0 & | & 0 \end{bmatrix} \tag{1.2.6.7}$$

Here, $\text{rank}(\mathbf{A}) = 1$ and $\text{rank}([\mathbf{A} \mid \mathbf{b}]) = 1$. Since $\text{rank}(\mathbf{A}) = \text{rank}([\mathbf{A} \mid \mathbf{b}]) < 2$, there exists exactly one value of k ($k = -1$) for which the system has an infinite number of solutions.
3. Condition for No Solution (Inconsistent System):
For no solution, the system must be inconsistent, which occurs when $\text{rank}(\mathbf{A}) \neq \text{rank}([\mathbf{A} \mid \mathbf{b}])$ (specifically $\text{rank}(\mathbf{A}) = 1$ and $\text{rank}([\mathbf{A} \mid \mathbf{b}]) = 2$).
This requires $1 - k^2 = 0$ while $-1 - k \neq 0$:

$$1 - k^2 = 0 \implies k = \pm 1 \tag{1.2.6.8}$$

$$-1 - k \neq 0 \implies k \neq -1 \tag{1.2.6.9}$$

This gives $k = 1$. At $k = 1$, the row echelon form becomes:

$$\begin{bmatrix} 1 & 1 & | & 1 \\ 0 & 0 & | & -2 \end{bmatrix} \tag{1.2.6.10}$$

Here, $\text{rank}(\mathbf{A}) = 1$ while $\text{rank}([\mathbf{A} \mid \mathbf{b}]) = 2$. Since $\text{rank}(\mathbf{A}) \neq \text{rank}([\mathbf{A} \mid \mathbf{b}])$, there exists exactly one value of k ($k = 1$) for which the system has no solution.
Therefore, options (a) and (d) are correct.
The following code gives the following figures (Fig. 1.2.6.1, Fig. 1.2.6.2, and Fig. 1.2.6.3).

codes/solution.py

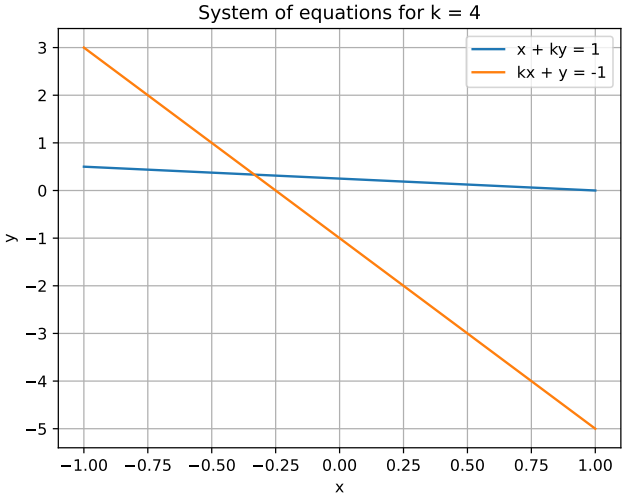


Fig. 1.2.6.1: System of linear equations intersection analysis

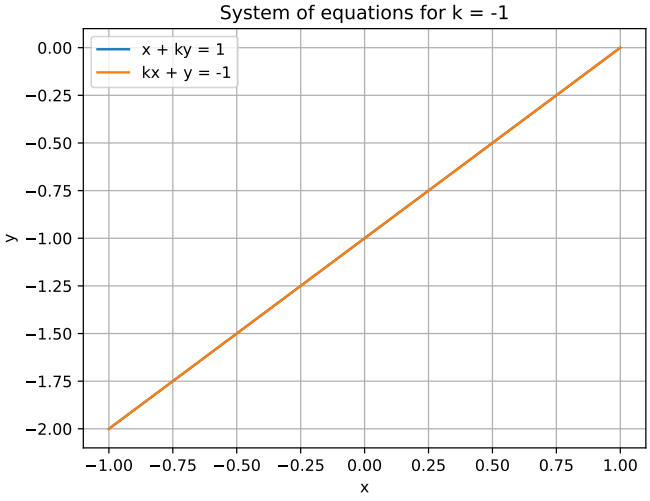


Fig. 1.2.6.2: System of equations for $k = -1$ (Coinciding lines)

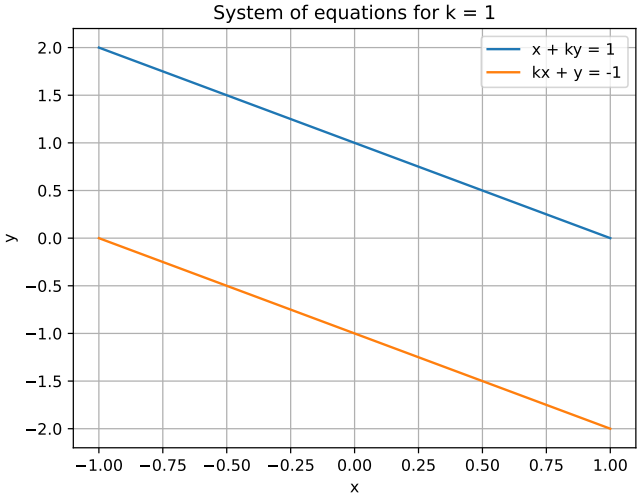


Fig. 1.2.6.3: System of equations for $k = 1$ (Parallel lines)

1.2.7 Let X be a 3-variable Boolean function that produces output as '1' when at least two of the input variables are '1'. Which of the following statement(s) is/are CORRECT, where a, b, c, d, e are Boolean variables? (CS1 2025)

a) $X(a, b, X(c, d, e)) = X(X(a, b, c), d, e)$
b) $X(a, b, X(a, b, c)) = X(a, b, c)$
c) $X(a, b, X(a, c, d)) = (X(a, b, a)$ AND $X(c, d, c))$
d) $X(a, b, c) = X(a, X(a, b, c), X(a, c, c))$

Solution: The function $X(a, b, c)$ produces output '1' whenever at least two of its three inputs a, b, c are '1'. The truth table describes this relationship:

a	b	c	$X(a, b, c)$
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Mapping the minterms $m(3,5,6,7)$ onto a 3-variable Karnaugh map (Fig. 1.2.7.1):

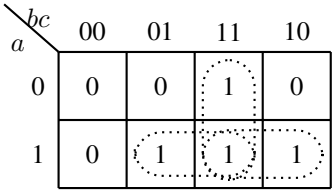


Fig. 1.2.7.1: K-map for 3-Variable Majority Function $X(a,b,c)$

Grouping adjacent '1's into pairs of size 2:

- Group 1 (cells m_3, m_7): bc
- Group 2 (cells m_5, m_7): ac
- Group 3 (cells m_6, m_7): ab

Summing these prime implicants gives the minimized majority Boolean function:

$$X(a,b,c) = ab + bc + ca \tag{1.2.7.1}$$

Evaluating each statement:

- Statement (a): $X(a,b,X(c,d,e)) = X(X(a,b,c),d,e)$
  Consider counterexample $a=0, b=0, c=1, d=1, e=1$:

$$\begin{aligned} \text{LHS} &= X(0,0,X(1,1,1)) && (1.2.7.2)\\ &= X(0,0,1) && (1.2.7.3)\\ &= 0 && (1.2.7.4)\\ \text{RHS} &= X(X(0,0,1),1,1) && (1.2.7.5)\\ &= X(0,1,1) && (1.2.7.6)\\ &= 1 && (1.2.7.7) \end{aligned}$$

  Since LHS $\neq$ RHS, statement (a) is incorrect.
- Statement (b): $X(a,b,X(a,b,c)) = X(a,b,c)$
  Substituting $X(a,b,c) = ab + bc + ca$:

$$\begin{aligned} X(a,b,X(a,b,c)) &= a\cdot b + b\cdot(ab+bc+ca) + a\cdot(ab+bc+ca) && (1.2.7.8)\\ &= ab + (ab+bc+abc) + (ab+abc+ca) && (1.2.7.9)\\ &= ab + bc + ca && (1.2.7.10)\\ &= X(a,b,c) && (1.2.7.11) \end{aligned}$$

  Hence, statement (b) is correct.
- Statement (c): $X(a,b,X(a,c,d)) = (X(a,b,a)$ AND $X(c,d,c))$

Evaluating each side:

$$\begin{aligned}
\text{RHS} &= X(a,b,a)\cdot X(c,d,c) && (1.2.7.12)\\
&= (a+ab)\cdot(c+cd) && (1.2.7.13)\\
&= a\cdot c && (1.2.7.14)\\
&= ac && (1.2.7.15)\\
\text{LHS} &= X(a,b,ac+ad+cd) && (1.2.7.16)\\
&= ab+b(ac+ad+cd)+a(ac+ad+cd) && (1.2.7.17)\\
&= ab+ac+ad+bcd && (1.2.7.18)
\end{aligned}$$

Since $ab+ac+ad+bcd \neq ac$ for general inputs, statement (c) is incorrect.

- Statement (d): $X(a,b,c) = X(a, X(a,b,c), X(a,c,c))$
  Note that $X(a,c,c) = ac+cc+ca = c$. Substituting into the RHS:

$$\begin{aligned}
\text{RHS} &= X(a, X(a,b,c), c) && (1.2.7.19)\\
&= a\cdot X(a,b,c) + c\cdot X(a,b,c) + ca && (1.2.7.20)\\
&= (a+c)(ab+bc+ca)+ca && (1.2.7.21)\\
&= (a^2b+abc+a^2c)+(abc+bc^2+c^2a)+ca && (1.2.7.22)\\
&= (ab+abc+ac)+(abc+bc+ac)+ac && (1.2.7.23)\\
&= ab+bc+ca && (1.2.7.24)\\
&= X(a,b,c) && (1.2.7.25)
\end{aligned}$$

Hence, statement (d) is correct.

Therefore, options (b) and (d) are correct.
The truth table is generated by the code below:

codes/truth_tab.c

1.2.8 The number -6 can be represented as 1010 in 4-bit 2's complement representation. Which of the following is/are CORRECT 2's complement representation(s) of -6? (CS1 2025)

a) 1000 1010 in 8-bits
b) 1111 1010 in 8-bits
c) 1000 0000 0000 1010 in 16-bits
d) 1111 1111 1111 1010 in 16-bits

Solution: For the 8-bit representation, $+6$ is expressed as $0000\,0110_2$. Flipping all the bits gives the 1's complement $1111\,1001_2$. Adding 1 yields the 2's complement:

$$1111\,1001_2 + 1 = 1111\,1010_2 \tag{1.2.8.1}$$

For the 16-bit representation, $+6$ is expressed as $0000\,0000\,0000\,0110_2$. Flipping all the bits gives the 1's complement $1111\,1111\,1111\,1001_2$. Adding 1 yields the 2's

complement:

$$1111\,1111\,1111\,1001_2 + 1 = 1111\,1111\,1111\,1010_2 \tag{1.2.8.2}$$

Therefore, options (b) and (d) are correct.

The following code verifies the solution:

codes/2comple.c

1.2.9 Let S be the set of all ternary strings defined over the alphabet $\{a, b, c\}$. Consider all strings in S that contain at least one occurrence of two consecutive symbols, that is, "aa", "bb" or "cc". The number of such strings of length 5 that are possible is _____. (CS1 2025)

Solution: The total number of ternary strings of length 5 is:

$$N_{\text{total}} = 3^5 = 243 \tag{1.2.9.1}$$

Using (1.1.13.1), the number of ternary strings of length 5 with NO consecutive identical symbols is:

$$N_{\text{no_consec}} = 3 \times (3-1)^{5-1} \tag{1.2.9.2}$$
$$= 3 \times 2^4 \tag{1.2.9.3}$$
$$= 48 \tag{1.2.9.4}$$

Subtracting from the total gives the required count:

$$N_{\text{consecutive}} = 243 - 48 \tag{1.2.9.5}$$
$$= 195 \tag{1.2.9.6}$$

The following code verifies the solution:

codes/ter_str.py

1.2.10 Consider the given function $f(x)$:

$$f(x) = \begin{cases} ax + b, & \text{for } x < 1 \\ x^3 + x^2 + 1, & \text{for } x \geq 1 \end{cases}$$

If the function is differentiable everywhere, the value of b must be _____. (CS1 2025)

Solution: For continuity at $x = 1$, using (1.1.14.1), the left-hand limit, right-hand

limit, and function value $f(1)$ must be equal:

$$\lim_{x\to 1^-} f(x) = \lim_{x\to 1^-} (ax+b) = a+b \tag{1.2.10.1}$$

$$\lim_{x\to 1^+} f(x) = \lim_{x\to 1^+} (x^3+x^2+1) = 3 \tag{1.2.10.2}$$

$$f(1) = 1^3+1^2+1 = 3 \tag{1.2.10.3}$$

Equating the left-hand and right-hand limits gives:

$$a+b=3 \tag{1.2.10.4}$$

Differentiating $f(x)$ piecewise for $x \neq 1$:

$$f'(x) = \begin{cases} a, & x<1 \\ 3x^2+2x, & x>1 \end{cases} \tag{1.2.10.5}$$

By (1.1.14.2), for $f(x)$ to be differentiable at $x=1$, the left-hand derivative and right-hand derivative must be equal:

$$\lim_{x\to 1^-} f'(x) = a \tag{1.2.10.6}$$

$$\lim_{x\to 1^+} f'(x) = \lim_{x\to 1^+} (3x^2+2x) \tag{1.2.10.7}$$

$$= 3(1)^2+2(1) \tag{1.2.10.8}$$

$$= 5 \tag{1.2.10.9}$$

Equating the left-hand and right-hand derivatives gives:

$$a=5 \tag{1.2.10.10}$$

Substituting $a=5$ into (1.2.10.4):

$$5+b=3 \implies b=3-5 \tag{1.2.10.11}$$

$$\implies b=-2 \tag{1.2.10.12}$$

Therefore, the required value of b is -2.
The following code gives the following figure (Fig. 1.2.10.1).

codes/verify_31.py

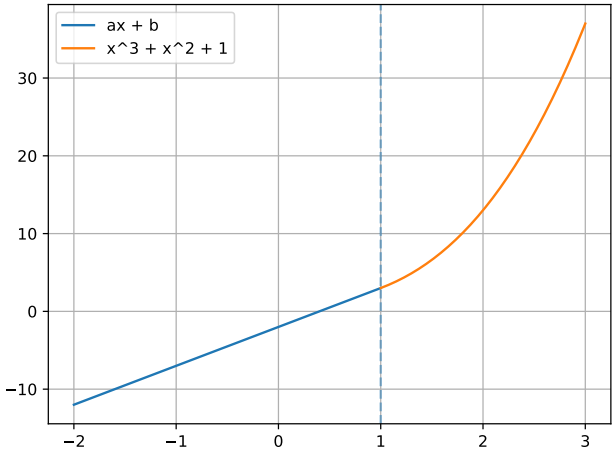


Fig. 1.2.10.1: Function differentiability plot

1.2.11 A box contains 5 coins: 4 regular coins and 1 fake coin. When a regular coin is tossed, the probability $P(\text{head}) = 0.5$ and for a fake coin, $P(\text{head}) = 1$. You pick a coin at random and toss it twice, and get two heads. The probability that the coin you have chosen is the fake coin is _____. (CS1 2025)

Solution: Let F be the event that the chosen coin is the fake coin, and R be the event that it is a regular coin. The prior probabilities of selecting each coin are $P(F) = \frac{1}{5}$ and $P(R) = \frac{4}{5}$. Let H_2 be the event of getting two heads when tossing the chosen coin twice.

The probability of getting two heads given a regular coin is:

$$P(H_2 \mid R) = (0.5)^2 = 0.25 \tag{1.2.11.1}$$

The probability of getting two heads given a fake coin is:

$$P(H_2 \mid F) = (1.0)^2 = 1.0 \tag{1.2.11.2}$$

Using Bayes' theorem (1.1.4.1):

$$P(F \mid H_2) = \frac{P(F)P(H_2 \mid F)}{P(F)P(H_2 \mid F) + P(R)P(H_2 \mid R)} \tag{1.2.11.3}$$

$$= \frac{\frac{1}{5} \times 1.0}{\left(\frac{1}{5} \times 1.0\right) + \left(\frac{4}{5} \times 0.25\right)} \tag{1.2.11.4}$$

$$= \frac{0.2}{0.2 + 0.2} \tag{1.2.11.5}$$

$$= 0.50 \tag{1.2.11.6}$$

Therefore, the probability that the chosen coin is the fake coin is 0.50.

1.2.12 The pseudocode of a function `fun()` is given below:

```
fun (int A[0,...,n−1]) {
    for i = 0 to n−2
        for j = 0 to n−i−2
            if (A[j] > A[j+1])
                then swap A[j] and A[j+1]
}
```

Let $A[0, \ldots, 29]$ be an array storing 30 distinct integers in descending order. The number of swap operations that will be performed, if the function `fun()` is called with $A[0, \ldots, 29]$ as argument, is _____. (CS1 2025)

Solution: The given pseudocode implements the standard Bubble Sort algorithm to sort an array in ascending order.
When an array of size $n = 30$ is given in strictly descending order, it represents the worst-case scenario where every adjacent pair comparison $A[j] > A[j+1]$ evaluates to true. Consequently, a swap operation is performed on every comparison.
Using (1.1.15.1):

$$\text{Total Swaps} = \frac{n(n-1)}{2} \tag{1.2.12.1}$$

$$= \frac{30 \times (30-1)}{2} \tag{1.2.12.2}$$

$$= \frac{30 \times 29}{2} \tag{1.2.12.3}$$

$$= 435 \tag{1.2.12.4}$$

Therefore, the total number of swap operations performed is 435.
The following code generates a random array A and performs the given function:

codes/verify_33.c

1.2.13 Consider the following C program:

```
#include <stdio.h>

void foo (int *p, int x) {
    *p = x;
}

int main() {
    int *z;
    int a = 20, b = 25;
```

```
    z = &a;
    foo (z, b);
    printf ("%d", a);
    return 0;
}
```

The output of the given C program is _____. (CS1 2025)

Solution: In `main()`, pointer `z` stores the address of variable `a`. Passing `z` to `foo()` allows `*p = x` to modify variable `a` directly to 25. Thus, `printf` outputs 25.

1.2.14 Let A be a 2×2 matrix as given:

$$A = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

What are the eigenvalues of the matrix A^{13}? (CS1 2025)

a) $1, -1$
b) $2\sqrt{2}, -2\sqrt{2}$
c) $4\sqrt{2}, -4\sqrt{2}$
d) $64\sqrt{2}, -64\sqrt{2}$

Solution: The characteristic equation of A is $\det(A - \lambda I) = (1-\lambda)(-1-\lambda) - 1 = \lambda^2 - 2 = 0 \implies \lambda = \pm\sqrt{2}$.
From (1.1.16.1):

$$\lambda^{13} = (\pm\sqrt{2})^{13} \tag{1.2.14.1}$$
$$= \pm(\sqrt{2})^{12} \cdot \sqrt{2} \tag{1.2.14.2}$$
$$= \pm 2^6\sqrt{2} \tag{1.2.14.3}$$
$$= \pm 64\sqrt{2} \tag{1.2.14.4}$$

The solution can be verified by the given code:

codes/eigenval.py

1.2.15 Consider the following four variable Boolean function in sum-of-product form:

$$F(b_3, b_2, b_1, b_0) = \Sigma(0, 2, 4, 8, 10, 11, 12)$$

where the value of the function is computed by considering $b_3b_2b_1b_0$ as a 4-bit binary number, where b_3 denotes the most significant bit and b_0 denotes the least significant bit. Note that there are no don't care terms. Which ONE of the following options is the CORRECT minimized Boolean expression for F? (CS1 2025)

a) $\bar{b}_1\bar{b}_0 + \bar{b}_2\bar{b}_0 + b_1\bar{b}_2b_3$
b) $\bar{b}_1\bar{b}_0 + \bar{b}_2\bar{b}_0$
c) $\bar{b}_2\bar{b}_0 + b_1b_2b_3$
d) $\bar{b}_0\bar{b}_2 + \bar{b}_3$

Solution: Truth Table for the 4-variable Boolean function $F(b_3, b_2, b_1, b_0) = \Sigma(0, 2, 4, 8, 10, 11, 12)$:

b_3	b_2	b_1	b_0	F
0	0	0	0	1
0	0	0	1	0
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0

b_3	b_2	b_1	b_0	F
1	0	0	0	1
1	0	0	1	0
1	0	1	0	1
1	0	1	1	1
1	1	0	0	1
1	1	0	1	0
1	1	1	0	0
1	1	1	1	0

Karnaugh Map (K-map) and Expression Minimization are shown in Fig. 1.2.15.1:

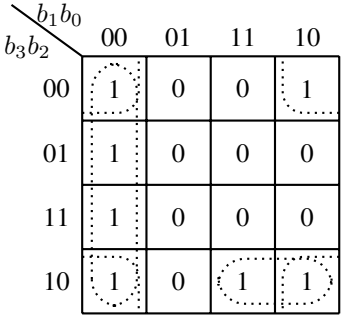


Fig. 1.2.15.1: K-map for 4-Variable Boolean Function $F(b_3, b_2, b_1, b_0)$

Grouping minterms into prime implicants:

- Group 1 (Column $b_1b_0 = 00$, minterms m_0, m_4, m_{12}, m_8): $\bar{b}_1\bar{b}_0$
- Group 2 (4 Corners, minterms m_0, m_2, m_8, m_{10}): $\bar{b}_2\bar{b}_0$
- Group 3 (Pair, minterms m_{10}, m_{11}): $b_3\bar{b}_2b_1$

Summing these prime implicants gives the minimized Boolean expression:

$$F = \bar{b}_1\bar{b}_0 + \bar{b}_2\bar{b}_0 + b_3\bar{b}_2b_1 \tag{1.2.15.1}$$

Therefore, option (a) is correct.
The following code generates the truth table:

codes/truth_tab_15.c

1.2.16 Let $G(V, E)$ be an undirected and unweighted graph with 100 vertices. Let $d(u, v)$ denote the number of edges in a shortest path between vertices u and v in V. Let the maximum value of $d(u, v)$, $u, v \in V$ such that $u \neq v$ be 30. Let T be any breadth-first-search tree of G. Which ONE of the given options is CORRECT for every such graph G? (CS1 2025)

a) The height of T is exactly 15.
b) The height of T is exactly 30.
c) The height of T is at least 15.
d) The height of T is at least 30.

Solution: Concept: The diameter of a graph is the maximum shortest path distance between any pair of vertices. For any breadth-first-search (BFS) tree T rooted at a vertex r, the distance from r to any vertex is at most the tree's height $h(T)$. By the triangle inequality, the distance between any two vertices u and v is bounded by $d(u, v) \leq d(r, u) + d(r, v) \leq 2h(T)$.
Solution: Given that the maximum shortest path distance in G is 30, from (1.1.17.1):

$$30 \leq 2h(T) \tag{1.2.16.1}$$

$$h(T) \geq 15 \tag{1.2.16.2}$$

Therefore, the height of T is at least 15, making option (c) correct.

1.2.17 $A = \{0, 1, 2, 3, \dots\}$ is the set of non-negative integers. Let F be the set of functions from A to itself. For any two functions, $f_1, f_2 \in F$ we define

$$(f_1 \odot f_2)(n) = f_1(n) + f_2(n)$$

for every number n in A. Which of the following is/are CORRECT about the mathematical structure $(F, \odot)$? (CS1 2025)

a) $(F, \odot)$ is an Abelian group.
b) $(F, \odot)$ is an Abelian monoid.
c) $(F, \odot)$ is a non-Abelian group.
d) $(F, \odot)$ is a non-Abelian monoid.

Solution: For the set of functions F mapping $A = \{0, 1, 2, 3, \dots\}$ to itself under the operation $(f_1 \odot f_2)(n) = f_1(n) + f_2(n)$:

- Closure and Associativity: The addition of non-negative integers is closed and associative, making $(F, \odot)$ an associative structure.
- Commutativity: Since $f_1(n)+f_2(n) = f_2(n)+f_1(n)$, the operation is commutative (Abelian).
- Identity Element: The zero function $e(n) = 0$ serves as the identity element since $f(n) + 0 = f(n)$.
- Inverses: Non-zero functions do not have inverse elements in F because the codomain consists solely of non-negative integers, meaning negative values required for additive inverses are absent.

Since it satisfies monoid axioms and commutativity without having inverses, $(F, \odot)$ is an Abelian monoid. Therefore, option (b) is correct.

1.2.18 Refer to the given 3-address code sequence. This code sequence is split into basic blocks. The number of basic blocks is _____. (CS1 2025)

```
1001: i = 1
1002: j = 1
1003: t1 = 10 * i
1004: t2 = t1 + j
1005: t3 = 8 * t2
1006: t4 = t3 - 88
1007: a[t4] = 0.0
1008: j = j + 1
1009: if j <= 10 goto 1003
1010: i = i + 1
1011: if i <= 10 goto 1002
1012: i = 1
1013: t5 = i - 1
1014: t6 = 88 * t5
1015: a[t6] = 1.0
1016: i = i + 1
1017: if i <= 10 goto 1013
```

Solution: To find the number of basic blocks, we first identify the code's leaders using (1.1.18.1):

- First Instruction: Instruction 1001 starts the program, so it is automatically a leader.
- Jump Targets: Instructions 1002, 1003, and 1013 are leaders because they are the destination targets of conditional jump (`goto`) statements in the loops.
- Instructions Following Jumps: Instructions 1010 and 1012 are leaders because they immediately follow jump instructions (lines 1009 and 1011), where control flow falls through when a loop condition becomes false.

Collecting these leaders gives the set $\{1001, 1002, 1003, 1010, 1012, 1013\}$.
Since there are 6 distinct leaders, the given 3-address code sequence is split into 6 basic blocks.

1.2.19 Consider the following database tables of a sports league:

- `player (pid, pname, age)`
- `coach (cid, cname)`
- `team (tid, tname, city, cid)`
- `members (pid, tid)`

An instance of the table and an SQL query are given below.

pid	pname	age
1	Jasprit	31

2	Atharva	24
3	Ishan	26
4	Axar	30

cid	cname
101	Ricky
102	Mark
103	Trevor

tid	tname	city	cid
10	MI	Mumbai	102
20	DC	Delhi	101
30	PK	Mohali	103

pid	tid
1	10
2	30
3	10
4	20

```
SELECT MIN (P.age)
FROM player P
WHERE P.pid IN (
    SELECT M.pid
    FROM team T, coach C, members M
    WHERE C.cname = 'Mark'
      AND T.cid = C.cid
      AND M.tid = T.tid
)
```

The value returned by the given SQL query is _____. (CS1 2025)

Solution: Coach 'Mark' has `cid = 102`, corresponding to team `tid = 10`. Players in team 10 are `pid = 1` (age 31) and `pid = 3` (age 26). The minimum age is $\min(31, 26) = 26$.

1.2.20 Suppose a 5-bit message is transmitted from a source to a destination through a noisy channel. The probability that a bit of the message gets flipped during transmission is 0.01. Flipping of each bit is independent of one another. The probability that the message is delivered error-free to the destination is _____. (CS1 2025)

Solution: To find the probability that the 5-bit message is delivered error-free, let $X_i \in \{0, 1\}$ be the random variable representing the number of errors in the i-th bit. Since the bit-flip probability is 0.01, the probability of a correct transmission is $1 - 0.01 = 0.99$. The corresponding PMF for a single bit is given by $P(X_i = 0) = 0.99$ and $P(X_i = 1) = 0.01$.
The probability generating function (Z-transform) for a single bit error distribution is defined using (1.1.2.1) as:

$$G_i(z) = E[z^{X_i}] \tag{1.2.20.1}$$
$$= P(X_i = 0)z^0 + P(X_i = 1)z^1 \tag{1.2.20.2}$$
$$= 0.99 + 0.01z \tag{1.2.20.3}$$

For 5 independent bits, the total number of errors is $Y = \sum_{i=1}^{5} X_i$. Applying the Z-transform property for sums of independent random variables (1.1.3.1), convolution corresponds to multiplying their generating functions:

$$G_Y(z) = \prod_{i=1}^{5} G_i(z) \tag{1.2.20.4}$$
$$= (0.99 + 0.01z)^5 \tag{1.2.20.5}$$

The probability of error-free delivery ($Y = 0$) corresponds to the constant term (coefficient of z^0) in $G_Y(z)$:

$$P(Y = 0) = [z^0]G_Y(z) \tag{1.2.20.6}$$
$$= (0.99)^5 \tag{1.2.20.7}$$
$$\approx 0.9510 \tag{1.2.20.8}$$

Therefore, the probability that the message is delivered error-free is approximately 0.95.
The following code gives the following figure (Fig. 1.2.20.1).

codes/errordistri.py

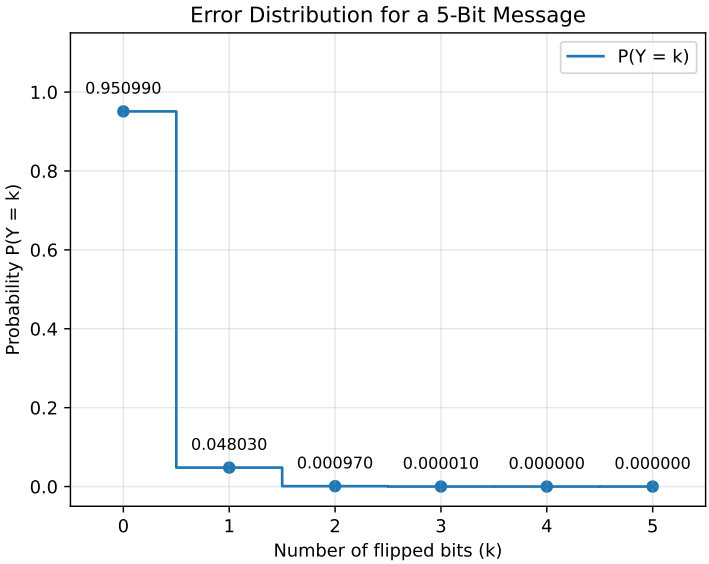


Fig. 1.2.20.1: Bit error probability distribution

1.2.21 Consider a probability distribution given by the density function $P(x)$:

$$P(x) = \begin{cases} Cx^2, & \text{for } 1 \leq x \leq 4 \\ 0, & \text{for } x < 1 \text{ or } x > 4 \end{cases}$$

The probability that x lies between 2 and 3, i.e., $P(2 \leq x \leq 3)$ is _____. (CS1 2025)

Solution: To find the probability using the Cumulative Distribution Function (CDF), we first determine the normalization constant C by integrating the probability density function over its domain using (1.1.19.1) and setting the total integral equal to 1:

$$\int_1^4 Cx^2\,dx = 1 \implies C\left[\frac{x^3}{3}\right]_1^4 = 1 \tag{1.2.21.1}$$

$$\implies C\left(\frac{64}{3} - \frac{1}{3}\right) = 1 \tag{1.2.21.2}$$

$$\implies C = \frac{1}{21} \tag{1.2.21.3}$$

The Cumulative Distribution Function $F_X(x) = P(X \leq x)$ represents the probability that the random variable X takes a value less than or equal to x. It is obtained by integrating the PDF $P(t)$ from the lower bound 1 up to x:

$$F_X(x) = \int_1^x \frac{t^2}{21}\,dt = \left[\frac{t^3}{63}\right]_1^x = \frac{x^3 - 1}{63} \quad \text{for } 1 \leq x \leq 4 \tag{1.2.21.4}$$

Thus, the complete piecewise CDF is defined as:

$$F_X(x) = \begin{cases} 0, & x < 1 \\ \dfrac{x^3 - 1}{63}, & 1 \le x \le 4 \\ 1, & x > 4 \end{cases} \tag{1.2.21.5}$$

Using the fundamental property of the CDF from (1.1.20.1), $P(a \le X \le b) = F_X(b) - F_X(a)$, the required probability that x lies between 2 and 3 is evaluated as the difference between the CDF values at the upper and lower boundaries:

$$P(2 \le X \le 3) = F_X(3) - F_X(2) \tag{1.2.21.6}$$

$$= \frac{3^3 - 1}{63} - \frac{2^3 - 1}{63} \tag{1.2.21.7}$$

$$= \frac{26}{63} - \frac{7}{63} \tag{1.2.21.8}$$

$$= \frac{19}{63} \approx 0.30158 \tag{1.2.21.9}$$

The following code gives the following figure (Fig. 1.2.21.1).

codes/pdf_21.py

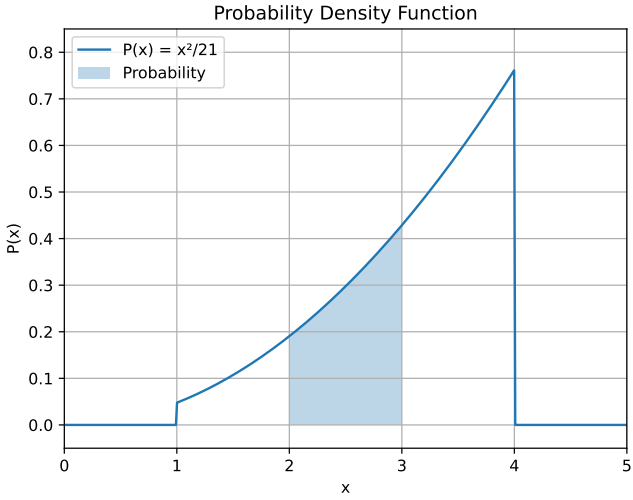


Fig. 1.2.21.1: Probability density function plot

The following code verifies the CDF plot by plotting random values and generates the following figure (Fig. 1.2.21.2).

codes/cdf_verify.py

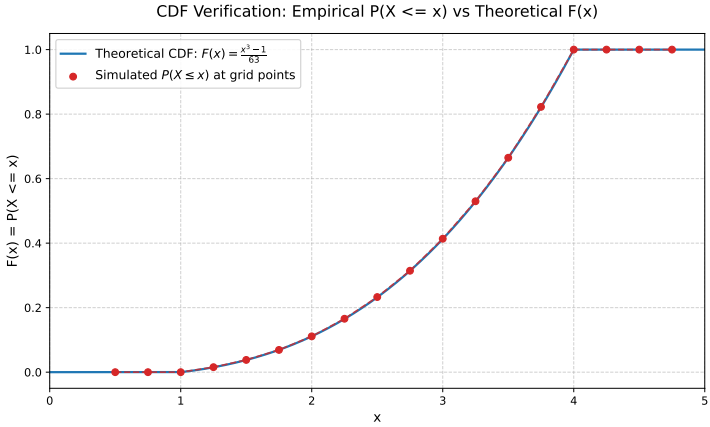


Fig. 1.2.21.2: Cumulative Distribution Function (CDF) plot

1.2.22 Consider the given finite state machine (FSM) with input X and output f. The minimum number of states required to realize the FSM is to be determined.

Present state	Next state		Output f	
	$X=0$	$X=1$	$X=0$	$X=1$
A	F	B	0	0
B	D	C	0	0
C	F	E	0	0
D	G	A	1	0
E	D	C	0	0
F	F	B	1	1
G	G	H	0	1
H	G	A	1	0

Solution:
Grouping states with identical output pairs gives initial partition:

$$P_0 = \{\{A, B, C, E\}, \{D, H\}, \{F\}, \{G\}\}.$$

Comparing their next-state groups splits $\{A, B, C, E\}$ into $\{A, C\}$ and $\{B, E\}$. The final minimal partition is:

$$P = \{\{A, C\}, \{B, E\}, \{D, H\}, \{F\}, \{G\}\}.$$

Hence, the minimized FSM has five states. Assign state codes using 3 D Flip-Flops $(Q_2Q_1Q_0)$:

$$S_0 = 000, \quad S_1 = 001, \quad S_2 = 010, \quad S_3 = 011, \quad S_4 = 100.$$

The state-transition diagram of the minimized FSM is shown in Fig. 1.2.22.1. Each transition is labelled by its input X.

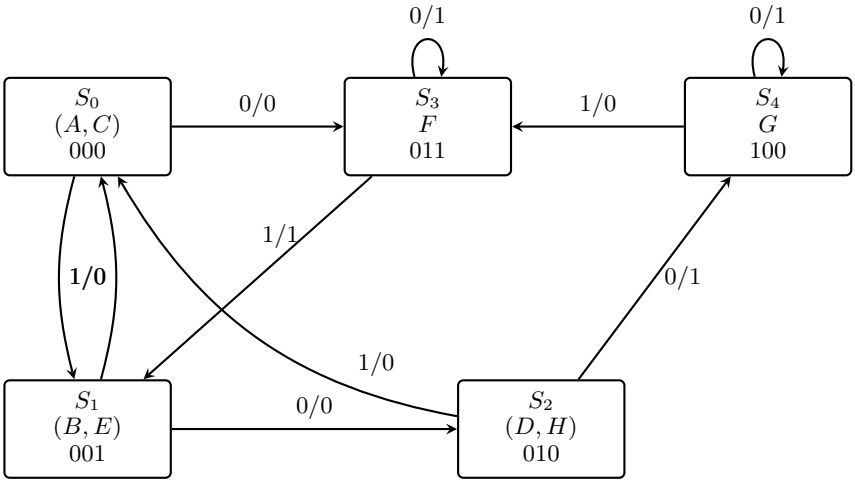


Fig. 1.2.22.1: State-transition Diagram of Minimized FSM

The minimized transition table, with separate rows for $X = 0$ and $X = 1$:

Present state	X	$Q_2Q_1Q_0$	Next state code	f
S_0	0	000	011	0
S_0	1	000	001	0
S_1	0	001	010	0
S_1	1	001	000	0
S_2	0	010	100	1
S_2	1	010	000	0
S_3	0	011	011	1
S_3	1	011	001	1
S_4	0	100	100	0
S_4	1	100	010	1

Using Karnaugh maps for D Flip-Flops excitation inputs (D_2, D_1, D_0) and output f (Fig. 1.2.22.2), with unused states $(101, 110, 111)$ as don't-care conditions (d):

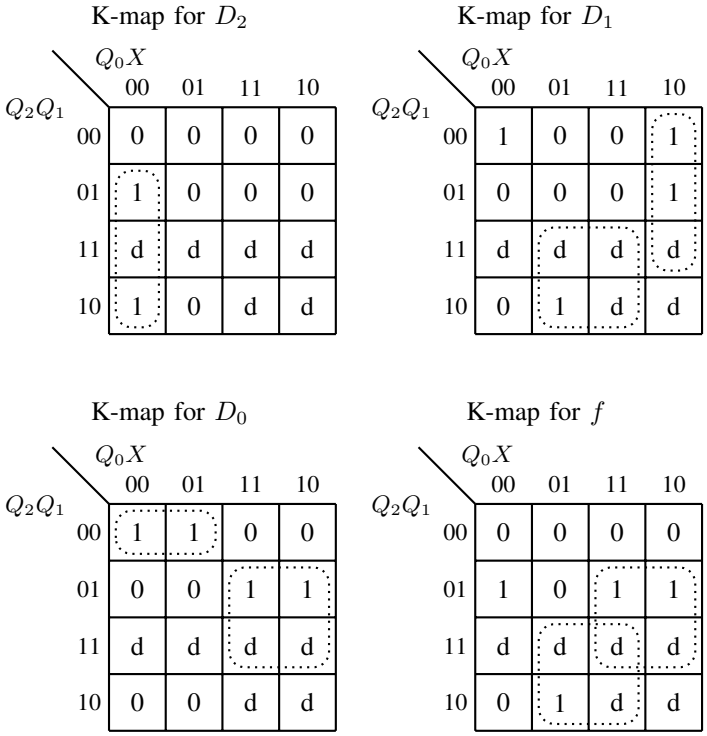


Fig. 1.2.22.2: Karnaugh maps for FSM excitation and output functions

From the Karnaugh maps, the minimized Boolean expressions for the next-state excitation inputs and output function are:

$$D_2 = \overline{X}Q_1\overline{Q_0} + \overline{X}Q_2 \tag{1.2.22.1}$$

$$D_1 = \overline{X}\,\overline{Q_2}\,\overline{Q_1} + \overline{X}Q_0 + XQ_2 \tag{1.2.22.2}$$

$$D_0 = \overline{Q_2}\,\overline{Q_1}\,\overline{Q_0} + Q_1Q_0 \tag{1.2.22.3}$$

$$f = Q_1\overline{X} + Q_1Q_0 + XQ_2 \tag{1.2.22.4}$$

Therefore, the minimum number of states required is 5.

Circuit Implementation Analysis (Fig. 1.2.22.3):

a) **Input Stage & State Storage:**

- The circuit receives external data input X and generates its complement X' ($\overline{X}$) using an inverter.
- State storage consists of three synchronous D Flip-Flops (`D Flip-Flop 1`, `D Flip-Flop 2`, and `D Flip-Flop 3`), driven by a shared common clock signal (`CLOCK CLK`).
- Their outputs represent state bits Q_2, Q_1, Q_0 alongside their complement signals Q_2', Q_1', Q_0' via vertical distribution rails.

b) **Combinational Output Logic (f):**

- Implements $f = Q_1\overline{X} + Q_1Q_0 + XQ_2$ using three 2-input AND gates driving a 3-input OR gate:

$$\text{AND}_1 = Q_1\overline{X} \tag{1.2.22.5}$$
$$\text{AND}_2 = Q_1Q_0 \tag{1.2.22.6}$$
$$\text{AND}_3 = XQ_2 \tag{1.2.22.7}$$

c) **Combinational Next-State Logic (D_2, D_1, D_0):**

- **For D_2:** Implements (1.2.22.1) using two 2-input AND gates ($\overline{X}Q_1\overline{Q_0}$ and $\overline{X}Q_2$) feeding a 2-input OR gate.
- **For D_1:** Implements (1.2.22.2) using three AND gates ($\overline{X}\,\overline{Q_2}\,\overline{Q_1}$, $\overline{X}Q_0$, and XQ_2) feeding a 3-input OR gate.
- **For D_0:** Implements (1.2.22.3) using a 3-input NOR gate ($\overline{Q_2 + Q_1 + Q_0} = \overline{Q_2}\,\overline{Q_1}\,\overline{Q_0}$) and a 2-input AND gate (Q_1Q_0) feeding a 2-input OR gate.

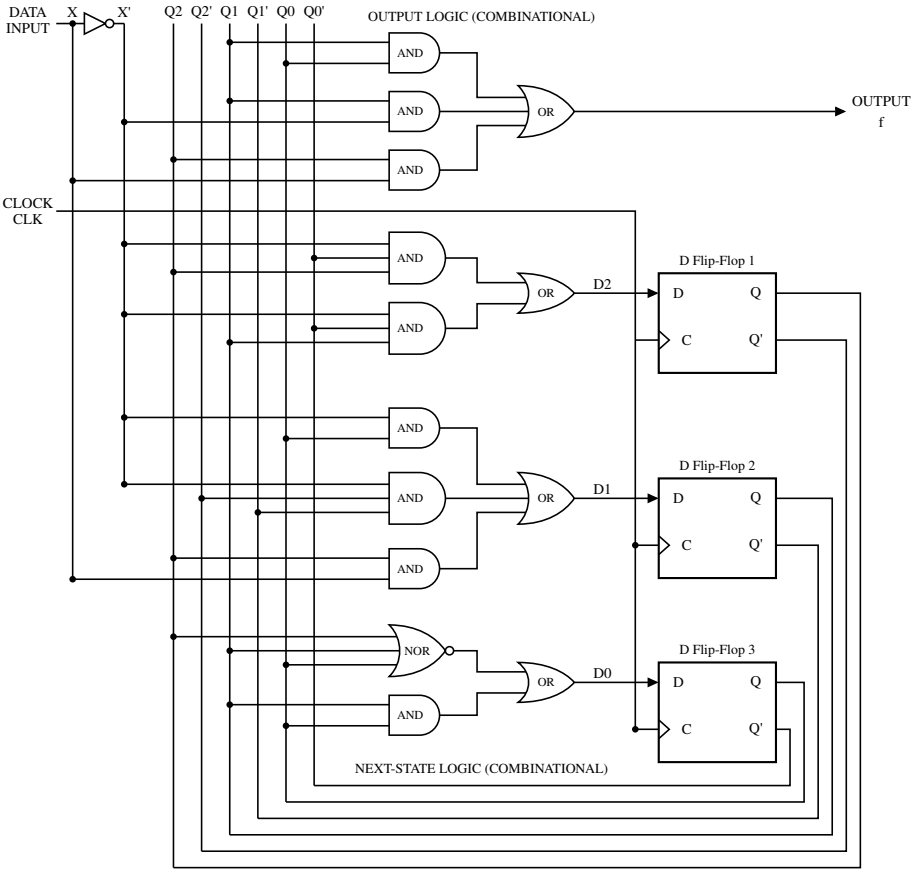
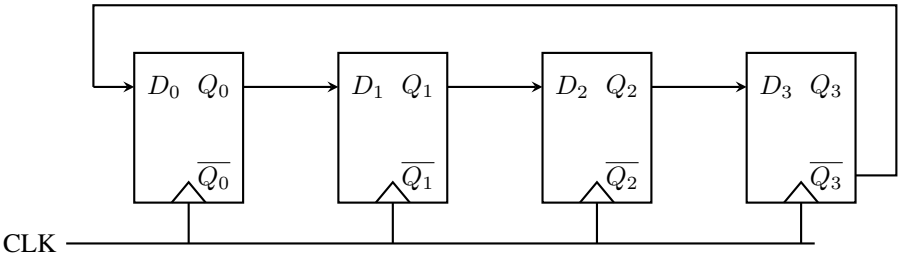


Fig. 1.2.22.3: FSM Sequential Circuit Implementation using D Flip-Flops

The following code verifies the solution:

codes/q22.c

1.2.23 Consider the given sequential circuit designed using D-Flip-flops. The circuit is initialized with some value (initial state). The number of distinct states the circuit will go through before returning back to the initial state is _____. (CS1 2025)



Solution: The given circuit is a 4-bit Johnson counter (twisted ring counter) constructed using four cascading D flip-flops, where the inverted output of the final stage is fed back to the first stage:

$$D_0 = \overline{Q_3}, \quad D_1 = Q_0, \quad D_2 = Q_1, \quad D_3 = Q_2 \tag{1.2.23.1}$$

Since $Q^+ = D$ for a D flip-flop, the next state function is:

$$(Q_0, Q_1, Q_2, Q_3)^+ = (\overline{Q_3}, Q_0, Q_1, Q_2) \tag{1.2.23.2}$$

Starting from an initial state of $(0, 0, 0, 0)$, the circuit progresses through the following sequence of states:

$$0000 \to 1000 \to 1100 \to 1110 \to 1111 \to 0111 \to 0011 \to 0001 \to 0000 \tag{1.2.23.3}$$

Therefore, the circuit goes through 8 distinct states before returning to the initial state.
The following code verifies the solution:

codes/q23.c

1.2.24 Consider the following C program:

```
#include <stdio.h>

int foo (int S[], int size) {
    if (size == 0) return 0;
```

```
    if (size == 1) return 1;
    if (S[0] != S[1]) return 1 + foo(S + 1, size − 1);
    return foo(S + 1, size − 1);
}

int main() {
    int A[] = {0, 1, 2, 2, 2, 0, 0, 1, 1};
    printf ("%d", foo(A, 9));
    return 0;
}
```

The value printed by the given C program is _____. (CS1 2025)

Solution: The function `foo(S, size)` recursively computes the total number of contiguous blocks (or runs) of identical elements in an array.
Mechanism of `foo(S, size)`:

- If $S[0] \neq S[1]$, a boundary between two distinct contiguous blocks is detected, so it adds 1 to the total count and recurses on $S + 1$ with size size $- 1$.
- If $S[0] = S[1]$, the adjacent elements belong to the same contiguous block, so it recurses on $S + 1$ without incrementing the count.
- The base case size $= 1$ returns 1, accounting for the final contiguous block.

Tracing the execution step-by-step for $A = \{0, 1, 2, 2, 2, 0, 0, 1, 1\}$ with size $= 9$:

$$\text{foo}(A, 9) = 1 + \text{foo}(A + 1, 8) \quad (A[0] \neq A[1], \text{ transition } 0 \to 1) \tag{1.2.24.1}$$

$$= 1 + 1 + \text{foo}(A + 2, 7) \quad (A[1] \neq A[2], \text{ transition } 1 \to 2) \tag{1.2.24.2}$$

$$= 1 + 1 + \text{foo}(A + 3, 6) \quad (A[2] = A[3] = 2) \tag{1.2.24.3}$$

$$= 1 + 1 + \text{foo}(A + 4, 5) \quad (A[3] = A[4] = 2) \tag{1.2.24.4}$$

$$= 1 + 1 + 1 + \text{foo}(A + 5, 4) \quad (A[4] \neq A[5], \text{ transition } 2 \to 0) \tag{1.2.24.5}$$

$$= 1 + 1 + 1 + \text{foo}(A + 6, 3) \quad (A[5] = A[6] = 0) \tag{1.2.24.6}$$

$$= 1 + 1 + 1 + 1 + \text{foo}(A + 7, 2) \quad (A[6] \neq A[7], \text{ transition } 0 \to 1) \tag{1.2.24.7}$$

$$= 1 + 1 + 1 + 1 + \text{foo}(A + 8, 1) \quad (A[7] = A[8] = 1) \tag{1.2.24.8}$$

At size $= 1$, the base case returns $\text{foo}(A + 8, 1) = 1$:

$$\text{foo}(A, 9) = 1 + 1 + 1 + 1 + 1 \tag{1.2.24.9}$$

$$= 5 \tag{1.2.24.10}$$

Therefore, the value printed by the program is 5.

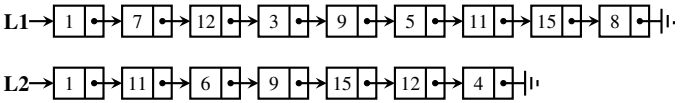
The following code generates a random A[] and then performs the given function on it:

codes/foo.c

1.2.25 Let `LIST` be a datatype for an implementation of linked list defined as follows:

```
typedef struct list {
    int data;
    struct list *next;
} LIST;
```

Suppose a program has created two linked lists, L1 and L2, whose contents are given in the figure below (code for creating L1 and L2 is not provided here). L1 contains 9 nodes, and L2 contains 7 nodes.



Consider the following C program segment that modifies the list L1. The number of nodes that will be there in L1 after the execution of the code segment is _____. (CS1 2025)

```
int find (int query, LIST *list) {
    while (list != NULL) {
        if (list->data == query) return 1;
        list = list->next;
    }
    return 0;
}

int main () {
    ptr1 = L1; ptr2 = L2;
    while (ptr1->next != NULL) {
        query = ptr1->next->data;
        if (find (query, L2))
            ptr1->next = ptr1->next->next;
        else
            ptr1 = ptr1->next;
    }
    return 0;
}
```

Solution: The code deletes every element in L1 (from 2nd node onward) that appears in L2. Elements $12, 9, 11, 15$ are deleted from L1. Retained elements are $[1, 7, 3, 5, 8]$, leaving 5 nodes in total.
The following code can verify the solution:

codes/q25.c

1.2.26 Consider the following C program:

```
#include <stdio.h>

int gate (int n) {
    int d, t, newnum, turn;
    newnum = turn = 0; t = 1;
    while (n >= t) t *= 10;
    t /= 10;
    while (t > 0) {
        d = n / t;
        n = n % t;
        t /= 10;
        if (turn) newnum = 10 * newnum + d;
        turn = (turn + 1) % 2;
    }
    return newnum;
}

int main () {
    printf ("%d", gate (14362));
    return 0;
}
```

The value printed by the given C program is _____. (CS1 2025)

Solution: The function `gate(n)` extracts the digits of n from left to right and constructs a new integer `newnum` using alternate digits, starting from the second most significant digit (`turn = 1`).
For input $n = 14362$:

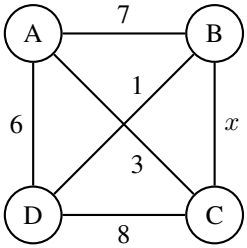
- The first `while` loop calculates $t = 10000$, matching the place value of the most significant digit.
- The second `while` loop extracts digits from left to right and toggles `turn` at each step:

Digit (d)	`turn`	Action	`newnum`	Next `turn`
1	0	Skip	0	1

4	1	Append 4	4	0
3	0	Skip	4	1
6	1	Append 6	46	0
2	0	Skip	46	1

After iterating through all digits, the function returns `newnum` $= 46$.
Therefore, the value printed by the program is 46.

1.2.27 The maximum value of x such that the edge between the nodes B and C is included in every minimum spanning tree of the given graph is _____. (CS1 2025)



Solution: By the cut property of minimum spanning trees, an edge is included in every MST if it is the strictly unique minimum-weight edge crossing some cut.
Consider the cut partitioning the vertices into $S = \{A, C\}$ and $V \setminus S = \{B, D\}$ where V is $\{A, B, C, D\}$. The crossing edges and their weights are:

- $(A, D) = 6$
- $(A, B) = 7$
- $(C, D) = 8$
- $(B, C) = x$

For edge (B, C) to be the unique minimum-weight edge across this cut, its weight must be strictly less than the minimum of the other crossing edges:

$$x < \min(6, 7, 8) \implies x < 6 \tag{1.2.27.1}$$

Since x must be an integer, the maximum value is $x = 5$.

1.2.28 In a double hashing scheme, $h_1(k) = k \bmod 11$ and $h_2(k) = 1 + (k \bmod 7)$ are the auxiliary hash functions. The size m of the hash table is 11. The hash function for the i-th probe in the open address table is $[h_1(k) + i \cdot h_2(k)] \bmod m$. The following keys are inserted in the given order: 63, 50, 25, 79, 67, 24. The slot at which key 24 gets stored is _____. (CS1 2025)

Solution: Using (1.1.21.1), the auxiliary functions for key 24 are:

$$h_1(24) = 24 \bmod 11 = 2 \tag{1.2.28.1}$$

$$h_2(24) = 1 + (24 \bmod 7) = 4 \tag{1.2.28.2}$$

Keys 63, 50, 25, 79, 67 occupy slots 8, 6, 3, 2, 1 respectively.

- $i = 0$: $h(24, 0) = 2 \implies$ slot 2 occupied.
- $i = 1$: $h(24, 1) = (2 + 4) \bmod 11 = 6 \implies$ slot 6 occupied.
- $i = 2$: $h(24, 2) = (2 + 8) \bmod 11 = 10 \implies$ slot 10 empty.

Key 24 is stored at slot 10.